

National University of Computer and Emerging Sciences



DL-2001: Introduction to Data Science Lab Manual 14

Department of Data Science
FAST-NU, Lahore, Pakistan

Objectives

- Understand **under fitting** and how it appears in real datasets.
- Learn to analyze **training vs. testing performance trends** using **learning curves**.
- Understand **over fitting** and how it appears in real datasets.

Model Selection:

Machine learning models aim to perform well on both training data and new, unseen data and is considered "good" if:

1. It learns patterns effectively from the training data.
2. It generalizes well to new, unseen data.
3. It avoids memorizing the training data (overfitting) or failing to capture relevant patterns (underfitting).

Bias and Variance in Machine Learning

Bias and variance are two key sources of error in machine learning models that directly impact their performance and generalization ability.

Bias: is the error that happens when a machine learning model is too simple and doesn't learn enough details from the data. It's like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.

- These assumptions make the model easier to train but may prevent it from capturing the underlying complexities of the data.
- High bias typically leads to **underfitting**, where the model performs poorly on both training and testing data because it fails to learn enough from the data.
- **Example:** A linear regression model applied to a dataset with a non-linear relationship.

Variance: Error that happens when a machine learning model learns too much from the data, including random noise.

- A high-variance model learns not only the patterns but also the noise in the training data, which leads to poor generalization on unseen data.
- High variance typically leads to **overfitting**, where the model performs well on training data but poorly on testing data.

Overfitting and Underfitting: The Core Issues

1. Overfitting in Machine Learning

Overfitting happens when a model learns too much from the training data, including details that don't matter (like noise or outliers).

- For example, imagine fitting a very complicated curve to a set of points. The curve will go through every point, but it won't represent the actual pattern.
- As a result, the model works great on training data but fails when tested on new data.

Overfitting models are like students who memorize answers instead of understanding the topic. They do well in practice tests (training) but struggle in real exams (testing).

Reasons for Overfitting:

1. High variance and low bias.
2. The model is too complex.
3. The size of the training data.

2. Underfitting in Machine Learning

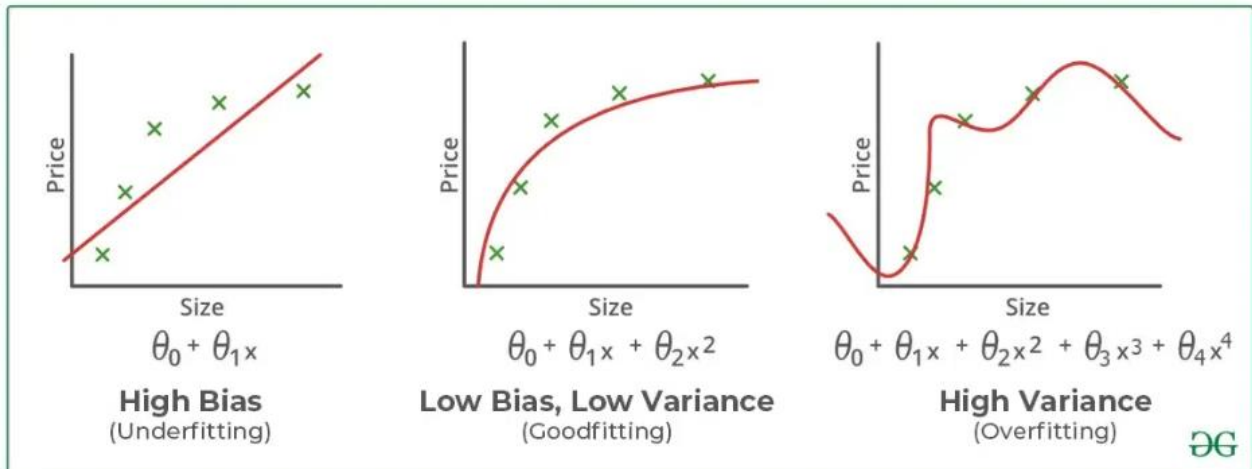
Underfitting is the opposite of overfitting. It happens when a model is too simple to capture what's going on in the data.

- For example, imagine drawing a straight line to fit points that actually follow a curve. The line misses most of the pattern.
- In this case, the model doesn't work well on either the training or testing data.

Underfitting models are like students who don't study enough. They don't do well in practice tests or real exams. **Note: The underfitting model has High bias and low variance.**

Reasons for Underfitting:

1. The model is too simple, So it may be not capable to represent the complexities in the data.
2. The input features which is used to train the model is not the adequate representations of underlying factors influencing the target variable.
3. The size of the training dataset used is not enough.
4. Excessive regularization are used to prevent the overfitting, which constraint the model to capture the data well.
5. Features are not scaled.



- **Underfitting** : Straight line trying to fit a curved dataset but cannot capture the data's patterns, leading to poor performance on both training and test sets.
- **Overfitting**: A squiggly curve passing through all training points, failing to generalize performing well on training data but poorly on test data.
- **Appropriate Fitting**: Curve that follows the data trend without overcomplicating to capture the true patterns in the data.

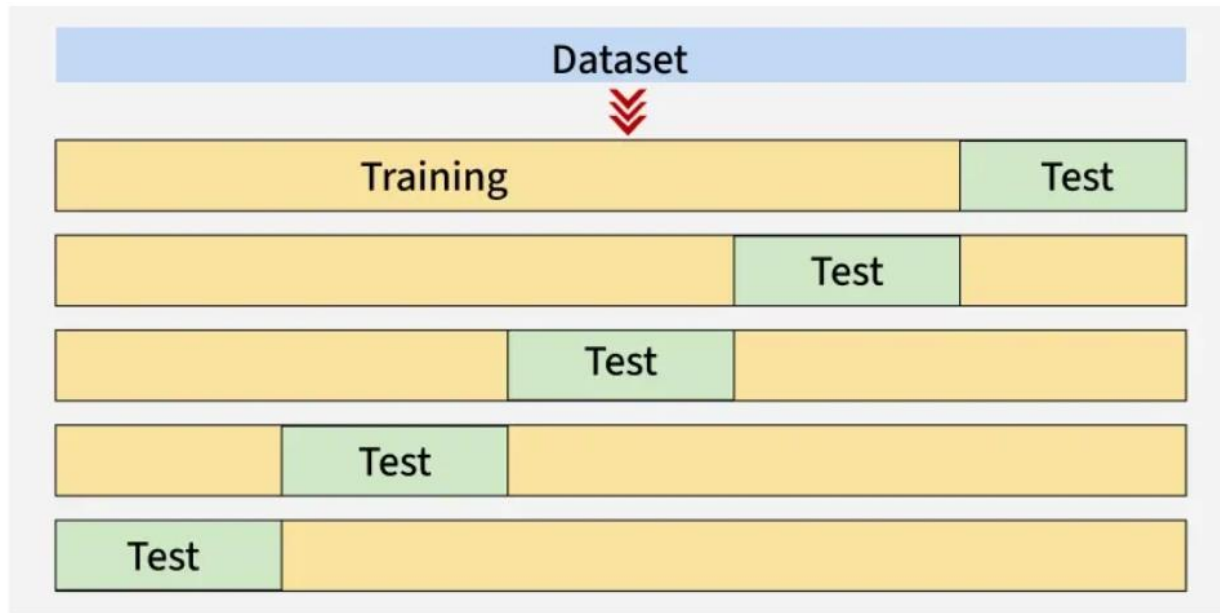
Cross - validation

Cross-validation is a technique used to check how well a machine learning model performs on unseen data while preventing overfitting. It works by:

- Splitting the dataset into several parts.
- Training the model on some parts and testing it on the remaining part.
- Repeating this resampling process multiple times by choosing different parts of the dataset.
- Averaging the results from each validation step to get the final performance.

K-Fold Cross Validation

It splits the dataset into k equal-sized folds. The model is trained on $k-1$ folds and tested on the remaining fold. This process is repeated k times each time using a different fold for testing. The diagram below shows an example of the training subsets and evaluation subsets generated in k -fold cross-validation. Here we have total 25 instances.



Here we will take k as 5.

- **1st iteration:** The first 20% of data [1–5] is used for testing and the remaining 80% [6–25] is used for training.
- **2nd iteration:** The second 20% [6–10] is used for testing and the remaining data [1–5] and [11–25] is used for training.
- This process continues until each fold has been used once as the test set.

Iteration	Training Set Observations	Testing Set Observations
1	[5-24]	[0-4]
2	[0-4, 10-24]	[5-9]
3	[0-9, 15-24]	[10-14]
4	[0-14, 20-24]	[15-19]
5	[0-19]	[20-24]

Each iteration uses different subsets for testing and training, ensuring that all data points are used for both training and testing.

References:

<https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>

<https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/>

Exercises

Step 1

- Use the built-in **California Housing Dataset** from `sklearn.datasets`.
- Load and split the dataset.

Step 2:

- Train a **Linear Regression** model on the training data.
- What is the **training MSE**? What is the **testing MSE**?
- Generate **learning curve** for the Linear Regression model. (explore this yourself)
- Describe the trend of the **training curve**.
- Describe the trend of the **testing curve**.
- Does this model appear to underfit, overfit, or fit well?

Step 3:

- Create a **Polynomial Regression model** (degree 5). (explore this yourself, use builtin libraries)
- Train the model on the training data.
- Predict on **testing** sets.

- What is the **training MSE**?
- What is the **testing MSE**?
- Generate a **learning curve** for the Polynomial Regression (degree 5) model.
- Describe the trend of both curves.
- How does the performance compare with Linear Regression?

Step 4:

Repeat above steps for **Polynomial Regression model** (degree 10).

Step 5:

- Which model has **highest training and testing error**?
- Which model shows **overfitting** (low training error, high testing error)?
- Which model shows underfitting?
- Which model has **good generalization**?

Task 2:

- Load Dataset
 1. Use the Diabetes dataset from `sklearn.datasets.load_diabetes()`.
 2. Separate features (X) and target (y).
- Set Up K-Fold Cross Validation (Explore this yourself. Use builtin libraries.)
 1. Use KFold with $k = 5$ and shuffle the data.
- Train KNN Model
 1. Train a Linear Regression model on the training folds.
 2. Predict on the test fold and calculate Mean Squared Error (MSE) for each fold.
- Calculate the average MSE across all folds.